CHANDIGARH UNIVERSITY
Discover. Learn. Empower.

# Experiment – 6

**Name: Probal Dhali** | **UID:** 24BAI71013
**Branch:** CSE (AIML) CS221 | **Section/Group:** 24AIT_NTPP-3 [B]
**Semester:** 5th | **Date of Performance:**
**Subject:** Full Stack - II | **Subject Code:** 24CSP-337

## EXPERIMENT – 6.1

### 1. AIM:

To implement paginated and sortable REST APIs for efficient and scalable data retrieval using Spring Boot and Spring Data JPA.

### 2. OBJECTIVE:

1. To understand the concepts of pagination and sorting in REST APIs.
2. To implement pageable APIs using Spring Data JPA.
3. To optimize data retrieval when handling large datasets.
4. To improve API performance and usability by retrieving data in smaller pages.
5. To implement sorting based on different student attributes.
6. To understand the use of Pageable, PageRequest, Page, and Sort in Spring Boot.

### 3. SOFTWARE REQUIREMENTS:

1. Java JDK 17 or above
2. Spring Boot
3. Visual Studio Code / IntelliJ IDEA / Eclipse
4. Maven
5. H2 Database
6. Postman or Web Browser for API Testing

### 4. THEORY

Large datasets should not be retrieved at once because this increases server load and response time.

Pagination divides data into smaller pages.
Example:
Page 1 → Records 1–5
Page 2 → Records 6–10

Sorting arranges records based on fields such as name or age in ascending or descending order.

Spring Data JPA provides:

- Pageable
- PageRequest
- Page
- Sort

Benefits:

- Faster response
- Reduced server load
- Better scalability

- Improved user experience

## 5. IMPLEMENTATION/PROCEDURE

- Create a Spring Boot project.
- Create the Student entity.
- Create StudentRepository.
- Use Pageable for pagination.
- Use Sort for sorting.
- Create Service and Controller layers.
- Add preloaded student data.
- Test the API using browser or Postman.

## 6. CODE

Student.java

```
package com.probal.pagination_api.entity;
import jakarta.persistence.*;
@Entity
@Table(name = "students")
public class Student {
   @Id
   @GeneratedValue(strategy = GenerationType.IDENTITY)
   private Long id;
   private String name;
   @Column(unique = true)
   private String uid;
   private String department;
   private Integer year;
   private Integer age;
   private String universityName;
   public Student() {
   }
   public Student(String name, String uid, String department,
            Integer year, Integer age, String universityName) {
      this.name = name;
      this.uid = uid;
      this.department = department;
      this.year = year;
      this.age = age;
      this.universityName = universityName;
   }
   public Long getId() {
      return id;
   }
   public String getName() {
      return name;
   }
   public String getUid() {
      return uid;
   }
```

```
    public String getDepartment() {
        return department;
    }
    public Integer getYear() {
        return year;
    }
    public Integer getAge() {
        return age;
    }
    public String getUniversityName() {
        return universityName;
    }
    public void setName(String name) {
        this.name = name;
    }
    public void setUid(String uid) {
        this.uid = uid;
    }
    public void setDepartment(String department) {
        this.department = department;
    }
    public void setYear(Integer year) {
        this.year = year;
    }
    public void setAge(Integer age) {
        this.age = age;
    }
    public void setUniversityName(String universityName) {
        this.universityName = universityName;
    }
}
```

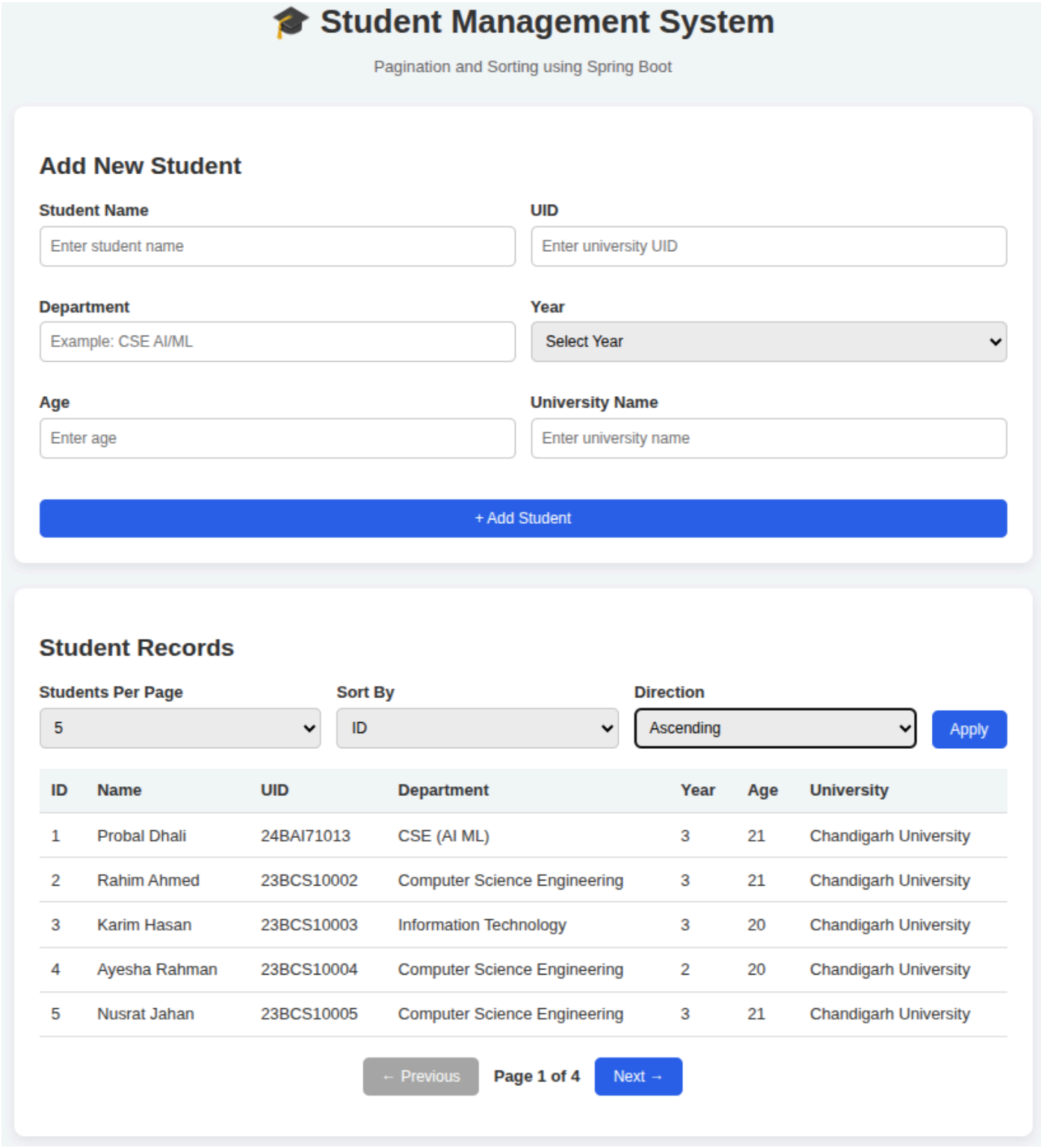
## Pagination Logic

```
Sort sort = Sort.by(sortBy);
if (direction.equalsIgnoreCase("desc")) {
    sort = sort.descending();
}
Pageable pageable = PageRequest.of(page, size, sort);
return studentRepository.findAll(pageable);
```

## API EXAMPLE

http://localhost:8080/api/students?page=0&size=5
With sorting:
http://localhost:8080/api/students?page=0&size=5&sortBy=name&direction=asc

## 7. OUTPUT




- Fig 6.1:REST API Student Management System

A Student Management System built using Spring Boot that allows users to add and manage student records. The system supports pagination and sorting, enabling efficient viewing of large datasets with customizable page size, sorting fields, and order.

## 8. LEARNING OUTCOME

- Pagination using Pageable.
- Sorting using Sort.
- Efficient retrieval of large datasets.
- REST API optimization.
- Spring Boot and Spring Data JPA integration.

CHANDIGARH UNIVERSITY
Discover. Learn. Empower.

# EXPERIMENT – 6.2

## 1. AIM:

To optimize backend read operations using caching and efficient database query strategies.

## 2. OBJECTIVE:

1. To identify inefficient database queries.
2. To understand and resolve the N+1 query problem.
3. To implement query optimization using JOIN FETCH.
4. To implement caching mechanisms.
5. To improve API performance and response time.

## 3. SOFTWARE REQUIREMENTS:

1. Java JDK 17+
2. Spring Boot
3. Spring Data JPA
4. H2 / MySQL / PostgreSQL Database
5. VS Code / IntelliJ IDEA
6. Apache JMeter for performance testing

## 4. THEORY

Backend performance can decrease when an application repeatedly accesses the database or executes inefficient queries.

The N+1 Query Problem occurs when one query retrieves main data and additional queries are executed for each related entity.

JOIN FETCH solves this problem by retrieving related entities in a single optimized query.

Caching stores frequently accessed data temporarily in memory, reducing repeated database access and improving response time.

Native SQL queries provide more control for complex database operations.

These techniques improve backend performance, scalability, and database efficiency.

## 5. IMPLEMENTATION/PROCEDURE

1. Create Department and Student entities.
2. Establish a One-to-Many and Many-to-One relationship.
3. Identify the N+1 query problem.
4. Implement JOIN FETCH for optimized data retrieval.
5. Create native SQL queries for specific operations.
6. Implement caching for frequently accessed data.
7. Test API performance before and after optimization.
8. Benchmark the API using Apache JMeter.

## 6. CODE

**Student.java**

```
package com.probal.backend_optimization_api.entity;
import jakarta.persistence.*;
@Entity
```

```
public class Student {
    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;
    private String name;
    @Column(unique = true, nullable = false)
    private String uid;
    private Integer age;
    @ManyToOne(fetch = FetchType.LAZY)
    @JoinColumn(name = "department_id")
    private Department department;
    public Student() {
    }
    public Student(
            String name,
            String uid,
            Integer age,
            Department department
    ) {
        this.name = name;
        this.uid = uid;
        this.age = age;
        this.department = department;
    }
    public Long getId() {
        return id;
    }
    public String getName() {
        return name;
    }
    public String getUid() {
        return uid;
    }
    public Integer getAge() {
        return age;
    }
    public Department getDepartment() {
        return department;
    }
    public void setName(String name) {
        this.name = name;
    }

    public void setUid(String uid) {
        this.uid = uid;
    }
    public void setAge(Integer age) {
        this.age = age;
    }
    public void setDepartment(Department department) {
```

```
        this.department = department;
    }
}
```

## 7. OUTPUT

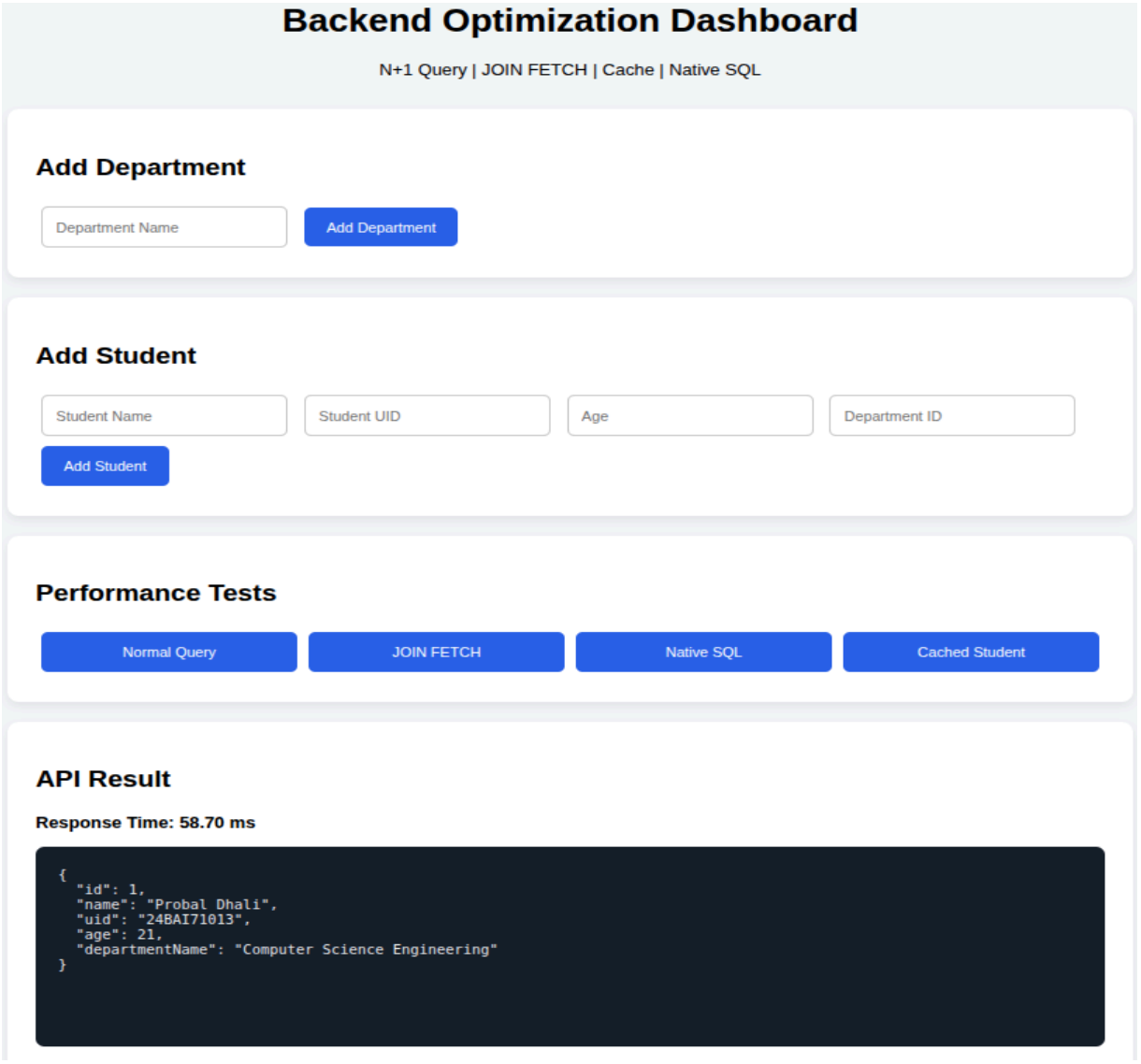



Fig 6.2: Backend Optimization Using Caching and Query Optimization

## 8. LEARNING OUTCOME

- The N+1 query problem.
- Query optimization using JOIN FETCH.
- Lazy loading in JPA.
- Native SQL queries.
- Backend caching concepts.
- API performance optimization.
- Backend scalability techniques.